

컴퓨터네트워크 소켓프로그래밍 Server, Client 프로그램 작성 과제

임시원 (20243106)

국민대학교 소프트웨어학부



02 동작 환경

공유기 포트 포워딩을 통해
외부에서 공유기를 통해
서버로 접속할 수 있도록
설계하였습니다.

메뉴탐색기

기본 설정

시스템 요약 정보

인터넷 설정 정보

무선 설정/보안

펌웨어 업그레이드

Easy Mesh

고급 설정

네트워크 관리

인터넷 설정 정보

내부 네트워크 설정

DHCP 서버 설정

무선랜 관리

NAT/라우터 관리

포트포워드 설정

고급 NAT 설정

라우팅 테이블 관리

보안 기능

특수기능

트래픽 관리

시스템 관리

UPN/서비스 관리

ipTIME A9004M-X2

다시 저장 도움

▼ 사용자정의 규칙보기

포트포워드 설정

순위	사용자 규칙	내부 IP	외부 포트	내부 포트	삭제
1	rpi	192.168.0.151	TCPUDP(1102~1102)	TCPUDP(80~80)	☐
2	rpiftp	192.168.0.151	TCPUDP(20~21)	TCPUDP(20~21)	☐
3	8080	192.168.0.237	TCPUDP(8080~8080)	TCPUDP(8080~8080)	☐
+ 새규칙 추가					

규칙이름

8080

포트포워드 사용자정의

☐ 규칙 비활성화

순위 3

▲ 순위높임

▼ 순위낮춤

내부 IP주소

192.168.0.237

☐ 현재 접속된 IP 주소

프로토콜

TCP/UDP

외부 포트

8080 ~ 8080

내부 포트

8080 ~ 8080

PC<-규칙저장

PC->규칙복원

파일 선택

선택된 파일 없음

새규칙

수정

취소

Mobile UI

02 Method-응답 HTTP 응답 코드 case 5개

번호	HTTP 상태 코드	상태 메시지	언제 사용되는지
1	200 OK	정상 처리 완료	- GET 요청으로 파일을 정상적으로 읽었을 때- PUT 요청으로 파일을 성공적으로 수정했을 때
2	100 Continue	계속 진행하세요	- HEAD 요청 시 파일이 존재할 때 (파일 존재만 확인)
3	404 Not Found	파일을 찾을 수 없음	- GET 요청 시 파일이 없을 때- HEAD 요청 시 파일이 없을 때- PUT 요청 시 파일이 없을 때
4	409 Conflict	충돌 발생 (이미 존재)	- POST 요청으로 파일을 만들려고 했는데 이미 파일이 있을 때
5	201 Created	파일 생성 완료	- POST 요청으로 새 파일을 성공적으로 생성했을 때

Server Code

02 코드 구조 (헤더)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <arpa/inet.h>
6  #include <sys/socket.h>
7  #include <sys/stat.h>
8  #include <fcntl.h>
```

코드 설명은 다음 페이지에 작성하였습니다

02 코드 구조 (헤더)

프로그램의 시작 부분에서는 서버를 구축하는 데 필요한 여러 표준 라이브러리들을 포함하였습니다.

우선 `stdio.h`를 포함하여 표준 입출력 함수인 `printf`, `perror` 등을 사용할 수 있도록 하였습니다.

다음으로 `stdlib.h`를 추가하여 프로그램 종료를 위한 `exit` 함수와 같은 기본 유틸리티 함수를 사용할 수 있도록 하였습니다.

문자열을 다루기 위해 `string.h`를 포함하였으며, 이를 통해 `strlen`, `strcmp`, `strcpy`와 같은 문자열 처리 함수를 사용할 수 있도록 하였습니다.

또한, 유닉스 시스템 호출을 지원하기 위해 `unistd.h`를 추가하였습니다. 이 파일을 통해 `read`, `write`, `close` 등의 시스템 호출 함수를 사용하였습니다.

네트워크 프로그래밍을 위해서는 `arpa/inet.h`를 포함하였습니다. 이 파일은 IP 주소를 처리하거나 소켓 주소를 변환하는 데 필요한 함수를 제공합니다.

이어 `sys/socket.h`를 포함하여 소켓 생성과 연결 요청 처리 등에 필요한 자료형과 함수를 사용할 수 있도록 하였습니다.

파일 상태를 확인하고 파일 정보를 얻기 위해 `sys/stat.h`를 포함하였습니다. 이 파일을 통해 `stat` 구조체와 관련 함수를 사용할 수 있게 하였습니다.

파일을 열거나 접근 방식을 설정하기 위해 `fcntl.h`를 추가하였습니다. 이를 통해 `open`, `O_RDONLY`, `O_WRONLY`, `O_CREAT` 등의 파일 제어 기능을 사용하였습니다.

이후에는 두 개의 상수 매크로를 정의하였습니다.

`BUF_SIZE`를 2048로 정의하여, 클라이언트로부터 요청을 받을 때 또는 파일을 읽고 쓸 때 사용할 버퍼의 크기를 2048바이트로 설정하였습니다.

또한, `PORT`를 8080으로 정의하여 서버가 클라이언트의 연결을 수신할 때 사용할 포트 번호를 8080번으로 설정하였습니다.

02 error_handling, send_response

```
13 void error_handling(char *message)
14 {
15     perror(message);
16     exit(1);
17 }
18
19 void send_response(int clnt_sock, char *status, char *content_type, char *body)
20 {
21     char buf[BUF_SIZE];
22     sprintf(buf, "HTTP/1.1 %s\r\nContent-Type: %s\r\nContent-Length: %ld\r\n\r\n%s",
23             status, content_type, strlen(body), body);
24     send(clnt_sock, buf, strlen(buf), 0);
25 }
```

코드 설명은 다음 페이지에 작성하였습니다

02 error_handling, send_response

먼저 error_handling 함수는 프로그램 실행 중 오류가 발생했을 때 호출되도록 작성하였습니다.

이 함수는 매개변수로 전달받은 문자열 message를 이용하여 perror 함수를 호출하였습니다.

perror 함수는 전달받은 메시지 뒤에 시스템이 설정한 오류 메시지를 함께 출력해주기 때문에, 디버깅 과정에서 오류의 원인을 쉽게 파악할 수 있도록 도와주었습니다.

오류 메시지를 출력한 뒤에는 exit(1)을 호출하여 프로그램을 비정상적으로 종료하도록 하였습니다.

exit 함수에 인자로 1을 전달함으로써, 호출자가 프로그램이 실패로 종료되었음을 알 수 있도록 처리하였습니다.

다음으로 send_response 함수는 클라이언트에게 HTTP 형식에 맞는 응답을 보내는 역할을 수행하였습니다.

이 함수는 네 개의 매개변수를 입력받도록 설계하였습니다.

clnt_sock은 응답을 전송할 클라이언트 소켓을 의미하며, status는 HTTP 응답 상태 코드와 메시지(예: "200 OK")를 전달받습니다.

또한 content_type은 응답 본문의 MIME 타입(예: "text/html", "text/plain")을 지정하는데 사용되었으며, body는 응답 본문의 실제 내용입니다.

함수 내부에서는 우선 BUF_SIZE 크기의 buf 배열을 선언하여 응답 메시지를 저장할 공간을 마련하였습니다.

그 다음, sprintf 함수를 사용하여 버퍼에 HTTP 응답 전체를 포매팅하여 작성하였습니다.

구체적으로, HTTP/1.1 <status> 줄을 시작으로 하여, Content-Type 헤더와 Content-Length 헤더를 작성한 뒤, 헤더와 본문을 구분하는 빈 줄(\r\n)을 추가하고 본문 내용을 덧붙였습니다.

이렇게 완성된 응답 문자열을 send 함수를 이용하여 클라이언트 소켓으로 전송하였습니다.

send 함수는 버퍼에 저장된 데이터를 지정된 소켓으로 송신하며, 여기서는 버퍼의 실제 길이만큼 전송하도록 strlen(buf)를 사용하였습니다.

02 handle_request

```
27 void handle_request(int clnt_sock, char *request)
28 {
29     char method[8], path[128];
30     sscanf(request, "%s %s", method, path);
31
32     char filepath[256] = "./files";
33     strcat(filepath, path);
34
35     if (strcmp(method, "GET") == 0)
36     {
37         int fd = open(filepath, O_RDONLY);
38         if (fd == -1)
39         {
40             send_response(clnt_sock, "404 Not Found", "text/plain", "File not found");
41         }
42         else
43         {
44             char body[BUF_SIZE];
45             read(fd, body, BUF_SIZE);
46             close(fd);
47             send_response(clnt_sock, "200 OK", "text/plain", body);
48         }
49     }
50     else if (strcmp(method, "HEAD") == 0)
51     {
52         struct stat st;
53         if (stat(filepath, &st) == -1)
54             send_response(clnt_sock, "404 Not Found", "text/plain", "");
55         else
56             send_response(clnt_sock, "100 Continue", "text/plain", "");
57     }
```

```
50     else if (strcmp(method, "HEAD") == 0)
51     {
52         struct stat st;
53         if (stat(filepath, &st) == -1)
54             send_response(clnt_sock, "404 Not Found", "text/plain", "");
55         else
56             send_response(clnt_sock, "100 Continue", "text/plain", "");
57     }
58     else if (strcmp(method, "POST") == 0)
59     {
60         int fd = open(filepath, O_WRONLY | O_CREAT | O_EXCL, 0644);
61         if (fd == -1)
62             send_response(clnt_sock, "409 Conflict", "text/plain", "File exists");
63         else
64         {
65             write(fd, "New file created by POST", 24);
66             close(fd);
67             send_response(clnt_sock, "201 Created", "text/plain", "File created");
68         }
69     }
70     else if (strcmp(method, "PUT") == 0)
71     {
72         int fd = open(filepath, O_WRONLY | O_TRUNC);
73         if (fd == -1)
74             send_response(clnt_sock, "404 Not Found", "text/plain", "File not found");
75         else
76         {
77             write(fd, "File modified by PUT", 20);
78             close(fd);
79             send_response(clnt_sock, "200 OK", "text/plain", "File updated");
80         }
81     }
82     else
83     {
84         send_response(clnt_sock, "400 Bad Request", "text/plain", "Invalid Method");
85     }
86 }
```

코드 설명은 다음 페이지에 작성하였습니다

02 handle_request

handle_request 함수는 클라이언트로부터 받은 HTTP 요청을 해석하고, 이에 따라 적절한 응답을 전송하는 기능을 수행하였습니다. 이 함수는 두 개의 매개변수를 받도록 구성되었습니다. 첫 번째 매개변수인 clnt_sock은 클라이언트와 통신하는 데 사용할 소켓 파일 디스크립터이며, 두 번째 매개변수인 request는 클라이언트가 전송한 HTTP 요청 메시지를 문자열 형태로 전달받습니다. 함수 내부에서는 우선 sscanf 함수를 이용하여 요청 메시지에서 HTTP 메서드와 요청 경로를 추출하였습니다.

method 배열에는 요청의 종류(GET, POST, PUT 등)가 저장되고, path 배열에는 클라이언트가 요청한 파일 경로가 저장되었습니다. 이후 파일 시스템상의 실제 경로를 생성하기 위해 filepath 배열을 초기화하였습니다.

filepath 배열은 기본 경로로 "./files"를 설정한 뒤, 클라이언트가 요청한 path를 이 경로에 덧붙이는 방식으로 구성하였습니다. 이를 통해 모든 파일 접근이 ./files 폴더 안에서만 이루어지도록 하여, 서버의 다른 영역에 대한 접근을 차단할 수 있도록 하였습니다.

클라이언트 요청의 메서드에 따라 각각 다음과 같은 처리를 수행하였습니다.

먼저, 요청 메서드가 "GET"인 경우에는 open 함수를 통해 요청된 파일을 읽기 전용으로 열었습니다.

파일 열기에 실패하면 "404 Not Found" 응답을 전송하였으며, 파일을 정상적으로 열 수 있다면 파일 내용을 read 함수를 이용하여 읽어들이고, 소켓을 통해 "200 OK" 응답과 함께 파일 내용을 클라이언트에 전송하였습니다.

파일을 읽은 이후에는 반드시 close 함수를 호출하여 파일 디스크립터를 닫아주었습니다.

다음으로, 요청 메서드가 "HEAD"인 경우에는 stat 함수를 사용하여 요청된 파일의 존재 여부를 확인하였습니다.

파일이 존재하지 않으면 "404 Not Found" 응답을 전송하였으며, 파일이 존재할 경우 "100 Continue" 응답을 보내어 파일의 존재를 알리도록 하였습니다.

02 handle_request

HEAD 메서드는 본문 없이 헤더 정보만 전달하는 요청이기 때문에, 본문 내용은 비워서 전송하였습니다.

요청 메서드가 "POST"인 경우에는 open 함수를 O_WRONLY | O_CREAT | O_EXCL 옵션과 함께 사용하여 새로운 파일을 생성하려 하였습니다.

만약 같은 이름의 파일이 이미 존재한다면 파일 생성에 실패하고 "409 Conflict" 응답을 전송하였습니다.

파일 생성에 성공한 경우에는 파일 안에 "New file created by POST"라는 내용을 작성한 뒤 파일을 닫고, "201 Created" 응답을 통해 파일이 성공적으로 생성되었음을 알렸습니다.

요청 메서드가 "PUT"인 경우에는 기존 파일을 열어서 수정하는 작업을 수행하였습니다.

open 함수에 O_WRONLY | O_TRUNC 옵션을 사용하여 파일을 열고, 기존 내용을 모두 지운 뒤 "File modified by PUT"라는 문자열을 새로 작성하였습니다.

파일 열기에 실패하면 "404 Not Found" 응답을 전송하였고, 성공적으로 수정이 완료되면 "200 OK" 응답을 전송하였습니다.

마지막으로, 위에 해당하지 않는 다른 메서드가 요청된 경우에는 "400 Bad Request" 응답을 보내어 클라이언트에게 잘못된 요청임을 알렸습니다.

이로써 서버는 GET, HEAD, POST, PUT 요청을 정상적으로 처리할 수 있으며, 알 수 없는 요청에 대해서도 명확하게 에러 처리를 수행할 수 있도록 구성되었습니다.

02 바인드, listen 부분

```
88 int main()
89 {
90     int serv_sock, clnt_sock;
91     struct sockaddr_in serv_addr, clnt_addr;
92     socklen_t clnt_addr_sz;
93     char buf[BUF_SIZE];
94
95     serv_sock = socket(PF_INET, SOCK_STREAM, 0);
96     if (serv_sock == -1)
97         error_handling("socket() error");
98
99     memset(&serv_addr, 0, sizeof(serv_addr));
100    serv_addr.sin_family = AF_INET;
101    serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
102    serv_addr.sin_port = htons(PORT);
103
104    if (bind(serv_sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) == -1)
105        error_handling("bind() error");
106
107    if (listen(serv_sock, 5) == -1)
108        error_handling("listen() error");
109
110    printf("Server listening on port %d...\n", PORT);
```

코드 설명은 다음 페이지에 작성하였습니다

02 바인드, listen 부분

main 함수는 서버 프로그램의 시작점으로, 서버 소켓을 초기화하고 클라이언트 요청을 수신하기 위한 준비를 수행하였습니다.

우선, 소켓 통신을 위해 필요한 변수들을 선언하였습니다.

serv_sock과 clnt_sock은 각각 서버 소켓과 클라이언트 소켓을 나타내는 정수형 변수로 선언하였습니다.

serv_addr과 clnt_addr은 서버와 클라이언트의 주소 정보를 저장하기 위한 sockaddr_in 구조체로 정의하였으며,

clnt_addr_sz는 클라이언트 주소 구조체의 크기를 저장하기 위해 socklen_t 타입으로 선언하였습니다.

또한 클라이언트 요청을 임시로 저장할 버퍼로 BUF_SIZE 크기의 buf 배열을 준비하였습니다.

서버 소켓은 socket 함수를 호출하여 생성하였습니다.

이때, 주소 체계는 PF_INET (IPv4 인터넷 프로토콜 패밀리)을 사용하였고, 소켓 타입은 SOCK_STREAM을 사용하여 TCP 통신을 선택하였습니다.

프로토콜 번호는 0을 지정하여 시스템이 자동으로 TCP를 선택하게 하였습니다.

소켓 생성에 실패한 경우 error_handling 함수를 호출하여 에러 메시지를 출력하고 프로그램을 종료하도록 하였습니다.

02 바인드, listen 부분

다음으로, 서버 소켓에 IP 주소와 포트 번호를 할당하기 위해 `serv_addr` 구조체를 초기화하였습니다. `memset` 함수를 사용하여 `serv_addr`의 모든 필드를 0으로 채운 뒤, 필요한 필드만 설정하였습니다. `sin_family` 필드는 주소 체계를 지정하기 위해 `AF_INET`으로 설정하였습니다. `sin_addr.s_addr` 필드는 `htonl(INADDR_ANY)`를 사용하여 서버가 모든 IP 주소(0.0.0.0)에서 오는 요청을 수신할 수 있도록 설정하였습니다.

또한, `sin_port` 필드는 `htons(PORT)`를 통해 지정된 포트 번호를 네트워크 바이트 오더로 변환하여 설정하였습니다. IP 주소와 포트 번호 설정이 완료된 후, `bind` 함수를 호출하여 서버 소켓에 주소 정보를 연결하였습니다. `bind` 호출이 실패하면 `error_handling` 함수를 호출하여 에러를 출력하고 프로그램을 종료하였습니다.

서버 소켓이 성공적으로 주소에 바인딩된 이후에는 `listen` 함수를 호출하여 클라이언트의 연결 요청을 대기 상태로 설정하였습니다. `listen` 함수의 두 번째 인자에는 대기열 크기(동시 수용 가능한 클라이언트 수)를 5로 지정하였습니다.

만약 `listen` 함수 호출에 실패하면 역시 `error_handling` 함수를 통해 에러 메시지를 출력하고 프로그램을 종료하도록 하였습니다. 서버가 정상적으로 준비를 마치고 클라이언트 요청을 받을 수 있는 상태가 되면, `printf` 함수를 이용하여 "Server listening on port 8080..." 이라는 안내 메시지를 터미널에 출력하였습니다.

이를 통해 사용자나 개발자는 서버가 정상적으로 동작 중이며, 포트 8080을 통해 외부 요청을 기다리고 있음을 알 수 있도록 하였습니다.

02 클라이언트의 요청 처리 loop

```
while (1)
{
    clnt_adr_sz = sizeof(clnt_addr);
    clnt_sock = accept(serv_sock, (struct sockaddr *)&clnt_addr, &clnt_adr_sz);
    if (clnt_sock == -1)
        continue;

    int str_len = read(clnt_sock, buf, BUF_SIZE - 1);
    buf[str_len] = '\0';
    handle_request(clnt_sock, buf);

    close(clnt_sock);
}

close(serv_sock);
return 0;
```

코드 설명은 다음 페이지에 작성하였습니다

02 클라이언트의 요청 처리 loop

서버가 클라이언트 요청을 처리하기 위해 본격적인 루프에 진입하는 부분은 while (1)로 무한 루프를 구성하여 구현하였습니다. 서버는 끊임없이 클라이언트의 연결을 기다리고 요청을 받아 처리할 수 있도록 설계하였습니다. 루프 안에서는 먼저 클라이언트 주소 구조체의 크기를 sizeof(clnt_addr)를 통해 계산하여 clnt_adr_sz에 저장하였습니다. 그 다음, accept 함수를 호출하여 클라이언트의 연결 요청을 수락하였습니다. accept 함수는 서버 소켓 serv_sock을 통해 연결 요청을 받아 새로운 클라이언트 소켓 clnt_sock을 반환하였습니다. 만약 accept 함수가 실패하여 -1을 반환하면, 에러 처리를 별도로 하지 않고 continue 문을 통해 현재 반복을 건너뛰고 다음 연결 요청을 기다리도록 하였습니다. 클라이언트와 연결이 성공적으로 수립되면, read 함수를 사용하여 클라이언트로부터 보낸 데이터를 읽어 들였습니다. 데이터는 최대 BUF_SIZE - 1 바이트까지 읽어왔으며, 읽은 데이터의 길이는 str_len 변수에 저장하였습니다. 읽은 데이터의 마지막에는 문자열 끝을 표시하기 위해 '\0'을 추가하여, 읽은 요청을 안전한 c 문자열로 만들었습니다.

02 클라이언트의 요청 처리 loop

이후, 클라이언트로부터 받은 요청 데이터를 기반으로 `handle_request` 함수를 호출하였습니다.
`handle_request` 함수는 클라이언트 소켓 번호와 읽어들이는 요청 데이터를 인자로 받아 요청을 분석하고 이에 맞는 HTTP 응답을 작성하여 클라이언트에게 전송하였습니다.
요청 처리가 완료된 후에는 `close(clnt_sock)`을 호출하여 클라이언트와의 연결을 종료하였습니다.
이렇게 함으로써 소켓 리소스를 적절히 해제하고 새로운 클라이언트 연결을 수용할 준비를 갖추었습니다.
루프가 계속 반복되기 때문에 서버는 다음 클라이언트 요청을 기다리게 되며, 이러한 방식으로 서버는 다수의 클라이언트 요청을 순차적으로 처리할 수 있었습니다.
프로그램이 종료되어야 할 경우를 대비하여, 루프를 벗어나면 서버 소켓 `serv_sock`도 `close` 함수를 호출하여 정상적으로 닫도록 하였습니다.
마지막으로 `return 0;`을 통해 `main` 함수는 0을 반환하고 프로그램을 정상적으로 종료할 수 있도록 하였습니다.



Client Code

02 헤더와 기본 설정

```
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <arpa/inet.h>
7 #include <sys/socket.h>
8
9 #define BUF_SIZE 2048
10 #define PORT 8080
11
12 void error_handling(char *message)
13 {
14     perror(message);
15     exit(1);
16 }
```

stdio.h를 포함하여 표준 입출력 함수인 printf, perror 등을 사용할 수 있도록 하였고, stdlib.h를 포함하여 프로그램 종료나 메모리 할당과 같은 일반 유틸리티 함수를 사용할 수 있도록 하였습니다. 문자열 처리를 위해 string.h를 포함하였으며, 이로써 strlen, strcpy, strcmp 같은 문자열 관련 함수들을 활용할 수 있게 하였습니다.

unistd.h를 포함함으로써 read, write, close 같은 시스템 호출 함수를 사용할 수 있도록 하였습니다. 네트워크 통신을 위해서는 arpa/inet.h를 포함하여 IP 주소 변환 함수인 inet_pton, inet_ntoa 등을 사용할 수 있도록 하였고, sys/socket.h를 포함하여 소켓 프로그래밍에 필요한 함수들과 자료형들을 사용할 수 있도록 준비하였습니다. 필요한 헤더 파일들을 포함한 이후, 코드에서는 매크로 상수를 정의하였습니다.

#define BUF_SIZE 2048을 선언하여, 서버와 통신할 때 사용할 버퍼 크기를 2048바이트로 고정하였습니다. 또한 #define PORT 8080을 선언하여, 클라이언트가 연결할 서버의 포트 번호를 8080번으로 지정하였습니다. 이 포트는 서버 프로그램과 동일하게 설정되어 있어야 정상적으로 연결이 이루어질 수 있습니다.

다음으로, 프로그램 전반에서 사용할 공통적인 오류 처리를 위한 함수를 작성하였습니다.

void error_handling(char *message) 함수는 시스템 호출이 실패하거나 예외적인 상황이 발생했을 때 호출되도록 하였습니다.

이 함수 내부에서는 먼저 perror(message)를 호출하여, 전달받은 메시지와 함께 시스템이 제공하는 상세한 오류 설명을 터미널에 출력하도록 하였습니다.

그리고 exit(1)을 호출하여, 프로그램을 비정상 종료시키고 운영체제에 오류가 발생했음을 알리도록 처리하였습니다.

02main함수와 request

```
18 int main(int argc, char *argv[])
19 {
20     if (argc != 3)
21     {
22         printf("Usage: %s <METHOD> <PATH>\n", argv[0]);
23         exit(1);
24     }
25
26     int sock;
27     struct sockaddr_in serv_addr;
28     char buf[BUF_SIZE];
29
30     sock = socket(PF_INET, SOCK_STREAM, 0);
31     if (sock == -1)
32         error_handling("socket() error");
33
34     memset(&serv_addr, 0, sizeof(serv_addr));
35     serv_addr.sin_family = AF_INET;
36     serv_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
37     serv_addr.sin_port = htons(PORT);
38
39     if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) == -1)
40         error_handling("connect() error");
41
42     sprintf(buf, "%s %s HTTP/1.1\r\n\r\n", argv[1], argv[2]);
43     write(sock, buf, strlen(buf));
44
45     int str_len = read(sock, buf, BUF_SIZE - 1);
46     buf[str_len] = '\0';
47     printf("Server Response:\n%s\n", buf);
48
49     close(sock);
50     return 0;
51 }
52
```

설명은 다음 페이지에 작성하였습니다

02 main 함수와 request

main 함수는 클라이언트 프로그램의 시작점으로서, 서버에 연결을 시도하고 HTTP 요청을 전송한 뒤 응답을 받아 출력하는 흐름을 수행하였습니다.

우선 프로그램이 실행될 때, 명령행 인자의 개수를 검사하였습니다.

argc가 3이 아니라면, 즉 프로그램 이름을 제외한 두 개의 인자(METHOD와 PATH)가 주어지지 않았다면,

printf를 이용하여 사용법을 출력하고, exit(1)을 호출하여 프로그램을 종료하도록 하였습니다.

이로써 클라이언트가 요청에 필요한 정보를 정확히 입력하지 않은 경우, 올바른 사용 방법을 안내할 수 있도록 하였습니다.

이후, 통신에 필요한 변수들을 선언하였습니다.

sock은 클라이언트 소켓 디스크립터를 저장하기 위한 정수형 변수로 선언하였고,

serv_addr은 서버의 주소 정보를 담기 위한 sockaddr_in 구조체로 선언하였습니다.

또한 서버로부터 받은 응답을 저장할 공간으로 BUF_SIZE 크기의 buf 배열을 준비하였습니다.

서버에 접속하기 위해 먼저 socket 함수를 호출하여 클라이언트 소켓을 생성하였습니다.

주소 체계로는 PF_INET을, 소켓 타입으로는 SOCK_STREAM을 지정하여 TCP 소켓을 생성하였고,

프로토콜은 0을 지정하여 시스템이 기본 TCP 프로토콜을 선택하도록 하였습니다.

소켓 생성에 실패할 경우, error_handling 함수를 호출하여 오류 메시지를 출력하고 프로그램을 종료하도록 하였습니다.

소켓이 정상적으로 생성되면, memset 함수를 사용하여 serv_addr 구조체의 모든 필드를 0으로 초기화하였습니다.

그 후 serv_addr.sin_family를 AF_INET으로 설정하여 IPv4 인터넷 프로토콜을 사용하도록 하였고,

serv_addr.sin_addr.s_addr에는 inet_addr("127.0.0.1")를 사용하여 서버 주소를 로컬호스트(127.0.0.1)로 설정하였습니다.

즉, 클라이언트는 자신의 컴퓨터에 실행 중인 서버에 연결을 시도하게 됩니다.

마지막으로 serv_addr.sin_port에는 htons(PORT)를 통해 포트 번호를 네트워크 바이트 순서로 변환하여 저장하였습니다.

서버 주소 설정이 완료되면, connect 함수를 호출하여 서버에 연결을 시도하였습니다.

연결에 실패할 경우, error_handling 함수를 호출하여 에러를 출력하고 프로그램을 종료하도록 하였습니다.

서버에 성공적으로 연결되면, 클라이언트는 서버에 전송할 HTTP 요청 메시지를 작성하였습니다.

sprintf 함수를 사용하여, 명령행 인자로 입력받은 METHOD와 PATH를 조합하여 HTTP 요청 메시지 포맷에 맞게 buf 배열에 작성하였습니다.

작성된 요청 메시지는 "METHOD PATH HTTP/1.1\r\n\r\n" 형태로 구성되어 있습니다.

이후, write 함수를 호출하여 소켓을 통해 서버로 요청 메시지를 전송하였습니다.

요청을 전송한 후에는 read 함수를 호출하여 서버로부터의 응답을 읽어 들였습니다.

최대 BUF_SIZE - 1 바이트만큼 읽어왔으며, 읽은 데이터의 끝에는 널 종료 문자 '\0'을 추가하여 C 문자열로 만들어주었습니다.

그 다음, printf 함수를 이용하여 "Server Response:"라는 제목과 함께 서버로부터 받은 응답 메시지를 터미널에 출력하였습니다.

마지막으로, 서버와의 통신이 끝나면 close(sock)을 호출하여 클라이언트 소켓을 닫고, return 0;을 통해 프로그램을 정상적으로 종료하였습니다.